

Visitors

February 4, 2026

Now that we've made an AST, we need to be able to do things with it.

That means we need to be able to traverse a tree.

Let's start with that.

Tree Traversal

Here's my definition for a Tree.

```
public class Tree
{
    public int x;
    public Tree left;
    public Tree right;
    ...
}
```

Now, I want to write a function add all of the numbers in the tree together.

Tree Traversal

This isn't too hard!

```
public int sum(Tree t)
{
    if(t == null) { return 0; }
    int left = sum(t.left);
    int right = sum(t.right);
    return t.x + left + right;
}
```

There are several software engineering concerns here,
but let's start with the most pressing.

I don't want **null**s in my tree!

Tree Traversal

Ok, that's easy to do.

```
abstract class Tree { ... }  
class Leaf extends Tree  
{  
    public int x;  
    ...  
}  
class Branch extends Tree  
{  
    public int x;  
    public Tree left;  
    public Tree right;  
    ...  
}
```

Now a tree is either a Leaf or a Branch.

Tree Traversal

Great, let's write that traversal algorithm!

Since I have 2 different classes, I can write a function for each class.

```
public int sum(Leaf l)
{
    return l.x;
}

public int sum(Branch b)
{
    int left = sum(b.left);
    int right = sum(b.right);
    return b.x + left + right;
}
```

This is much cleaner code!

Too bad it doesn't work.

Overriding vs Overloading

In order to solve this problem, we need to use *overriding*.

Let's review:

Overriding vs Overloading

Overloading is when 2 function take different arguments.

```
class A { ... }  
class B extends A { ... }  
  
class Main {  
    public void print(A a) {  
        System.out.println("A");  
    }  
    public void print(B b) {  
        System.out.println("B");  
    }  
    public static void main(String[] args) {  
        A b = new B();  
        print(b);  
    }  
}
```

What gets printed? (Hint: It'll be either "A" or "B")

Overriding vs Overloading

The answer is “A”.

Overloading

When A compiler decides what function to call,
It looks at the *declared* type of the argument.

Overriding vs Overloading

Overriding is when I replace an inherited method.

```
class A {  
    public void print() {  
        System.out.println("A");  
    }  
}  
class B extends A {  
    public void print() {  
        System.out.println("B");  
    }  
}  
class Main {  
    public static void main(String[] args) {  
        A b = new B();  
        b.print();  
    }  
}
```

What gets printed?

Overriding vs Overloading

The answer is “B”.

Overriding

When A compiler decides what method to call, it looks up the method *inside* the object. So, it uses the actual value of the object.

Don't worry, we will talk about how this happens *at length* later.

Tree Traversal

Back to our example.

We can solve this problem if we make sum a method of Tree

```
abstract class Tree { public abstract int sum(); ... }
class Leaf extends Tree {
    public int x;
    public int sum() {
        return x;
    }
}
class Branch extends Tree {
    public int x;
    public Tree left;
    public Tree right;
    public int sum() {
        return x + left.sum() + right.sum();
    }
}
```

This one actually works!

Traversing ASTs

Ok, so what's the big deal.

We'll just write all of our traversals as methods.

Well, we have 63 different classes in our AST.

That means to update a class, we need to touch 63 different files!

Trust me on this, you do not want to do this!

Traversing ASTs

The solution to this problem is *not* intuitive, but it is very effective.

We want to have our code grouped together in one class.
I'm going to call that class a *visitor*

```
public class Visitor {  
    public Object visit(Leaf l) { ... }  
    public Object visit(Branch b) { ... }  
}
```

But this is the problem we had before.
All Java can't know what type of Tree we have.

We need to call a method for our recursive call.

Traversing ASTs

Totally fine, we just add a method that takes a visitor.

```
public class Visitor
{
    public Object visit(Leaf l)
    {
        l.accept(this);
        return null;
    }
    public Object visit(Branch b)
    {
        b.left.accept(this);
        b.right.accept(this);
        return null;
    }
}
```

Traversing ASTs

Now the accept method can call the correct visitor!

```
public abstract class Tree {  
    public abstract Object accept(Visitor v);  
}  
  
public class Leaf extends Tree {  
    public Object accept(Visitor v) {  
        v.visit(this);  
    }  
}  
  
public class Branch extends Tree {  
    public Object accept(Visitor v) {  
        v.visit(this);  
    }  
}
```

But how do I know what visit method to call?
in the Branch class, `this` is a Branch object.

Using visitors

Ok, that's cool, but how do I actually use it?

Let's make a SumVisitor

```
public class SumVisitor extends Visitor {  
    public Object visit(Leaf l)  
    {  
        return l.x;  
    }  
    public Object visit(Branch b)  
    {  
        Integer left  = b.left.accept(this);  
        Integer right = b.right.accept(this);  
        return b.x + left + right;  
    }  
}
```

Using visitors

Let's compare the visitor with our original version

Original:

```
public int sum(Leaf l)
{
    return l.x;
}
public int sum(Branch b)
{
    int left = sum(b.left);
    int right = sum(b.right);
    return b.x + left + right;
}
```

Using visitors

Let's compare the visitor with our original version

Visitor:

```
public class SumVisitor extends Visitor {
    public Object visit(Leaf l)
    {
        return l.x;
    }
    public Object visit(Branch b)
    {
        Integer left  = b.left.accept(this);
        Integer right = b.right.accept(this);
        return b.x + left + right;
    }
}
```

Using visitors

Let's compare the visitor with our original version

```
public class SumVisitor extends Visitor {
    public Object visit(Leaf l)
    {
        return l.x;
    }
    public Object visit(Branch b)
    {
        Integer left  = b.left.accept(this);
        Integer right = b.right.accept(this);
        return b.x + left + right;
    }
}

public int sum(Leaf l)
{
    return l.x;
}
public int sum(Branch b)
{
    int left = sum(b.left);
    int right = sum(b.right);
    return b.x + left + right;
}
```

The Visitor is the same as any other recursive function.
We just replace the recursion with calls to accept

Traversing ASTs

How does this work with and AST?

Exactly the same.

```
public class If extends Stmt
{
    public Exp exp;
    public Stmt trueStmt;
    public Stmt falseStmt;
    ...
}
```

Traversing ASTs

The visitor for If just recursively looks at the children.

```
public class Visitor
{
    ...
    public Object visit(If n)
    {
        n.exp.accept(this);
        n.trueStmt.accept(this);
        n.falseStmt.accept(this);
        return null;
    }
    ...
}
```

Traversing ASTs

So, why use the visitor pattern?

- It lets us write all of our code in one place!
- Since we inherit from Visitor,
we don't have to fill in nodes that don't matter.

Let's see that last point in action. I want to write a visitor to find all of the variables used in a program.

A variable is

- Any local variable (IDExp)
- Any field

Traversing ASTs

```
public class VarVisitor {  
    private HashSet<String> vars = new HashSet<>();  
  
    public Object visit(IDExp n) {  
        vars.add(n.name);  
        return null;  
    }  
  
    public Object visit(FieldAccess n) {  
        vars.add(n.name);  
        exp.accept(this);  
        return null;  
    }  
}
```

I only had to write 2 methods to traverse the entire AST!

We are going to be using visitors for basically the rest of the class.

There are a few important points.

- Only define visitors for the classes you need.
 - If you don't need a value back, return `null`
 - If you do need a value back, make sure everyone returns one!
- This is just like writing any other recursive function.
- **Don't forget to call `accept()` on your children**